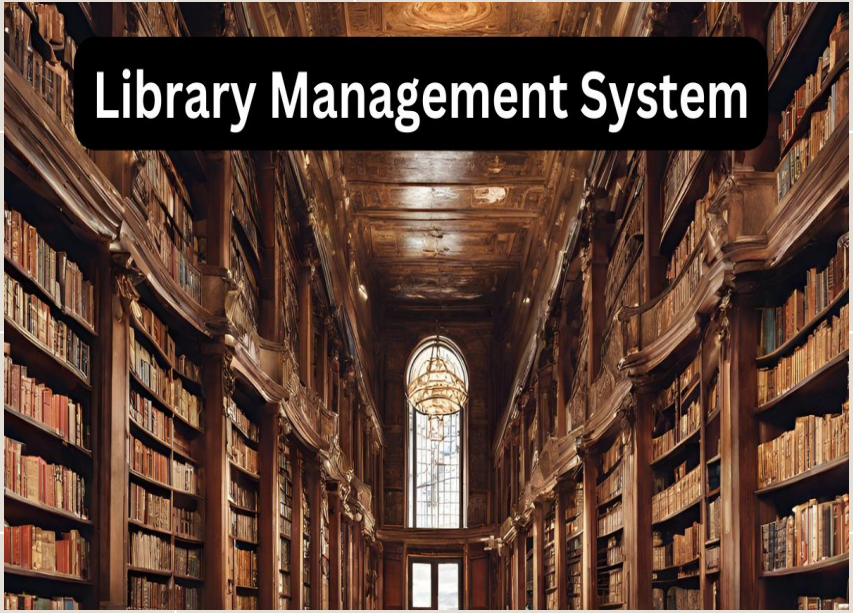
ITIS 3300

Library Management System

Brian Tramuel · Will Armentrout · Sam Suther · Isaac Vunzi · Sana Ullah

ITIS 3300 · Spring 2026

Team Members



1. 👮 Brian Tramuel
2. 👮 Sana Ullah
3. 👮 Isaac Vunzi
4. 👮 Sam Suther
5. 👮 Will Armentrout

🔗 GitHub: ssuther6-ops/ITIS3300-Panthers

Team Responsibilities

Backend API

Borrow/return endpoints, atomic transactions, borrowing limit enforcement, availability checks

Database Design

borrowing_transactions schema, PostgreSQL transaction wrapping (BEGIN/COMMIT/ROLLBACK), connection pooling

Frontend

Borrow/return button states, availability badges, real-time UI updates, placeholder loan list page and password strength indicator.

Testing

19 manual Postman test cases covering auth, catalog, borrow/return, edge cases, admin routes

Code Inspection

Reviewed feedback from Pintos team, addressed 4 flagged items magic numbers, path comments, falsy checks, color constants

Documentation

Phase 1 requirements, Phase 2 requirements, UML diagrams (Class, Sequence, Use-Case), User Manual, Compile/Run Instructions

Project Overview

What We Built

- Web-based Library Management System
- Node.js + Express backend
- PostgreSQL database
- HTML/CSS/JS frontend
- JWT-based session authentication
- RESTful API architecture

Project Phases

Phase 1
MVP Auth & Catalog - FR-1–8, NFR-1–4, NFR-9–10

Phase 2
Borrow, Return & Security - FR-9–16, NFR-5–7

Phase 3
Admin Tools & Polish - FR-17–23, NFR-8, NFR-11–12

Core Requirements - Functional (Phase 1)

FR-1

User Registration

Users provide a full name, a unique email, a unique username, and a password meeting complexity rules.

FR-2

Login / Logout

Registered users authenticate with username and password. A JWT is issued on success and stored client-side. Logout invalidates the token server-side.

FR-3

Role-based Access

Every account carries a role: User or Admin. The default at registration is User.

FR-4

Password Requirements

Passwords must be at least 8 characters and include one uppercase letter, one lowercase letter, one digit, and one special character.

FR-5

Browse Catalog

Any visitor can view a paginated list of active books. Each card shows title, author, genre, and an availability badge. Default sort is alphabetical by title.

FR-6

Search by Title / Author

A search bar accepts free-text queries. The backend runs a case-insensitive search on the title and author columns.

FR-7

Filter and Sort

Filters: availability and genre. Sort options: Title A-Z, Title Z-A, Author A-Z, Newest Added.

FR-8

Book Detail View

Clicking a book opens a detail page showing title, author, ISBN, genre, total copies, available copies, and status.

Core Requirements - Functional (Phase 2)

FR-9

Book Availability Status

Real-time availability badge on catalog & detail pages, no full reload required.

FR-10

Borrow a Book

Authenticated borrow; inserts transaction record, atomically decrements available_copies, returns due date.

FR-11

Loan Linked to Account

Each loan tied to user via foreign key; users cannot view or modify others' loans.

FR-12

Due Date Storage

Each loan stores a due_date for 14 days, the borrowed_at using a PostgreSQL default expressions. Returned in API response and shown on user's loan list.

FR-13

Return a Book

Atomically sets status='returned', records returned_at, increments available_copies.

FR-14

Prevent Borrowing Unavailable

Checks available_copies; returns HTTP 400 'No copies available'. Frontend disables Borrow button.

FR-15

View Borrowed Books

GET /api/borrow/my, shows title, author, borrowed_at, due_date, and current status for aactive, returned, and overdue.

FR-16

Borrowing Limit

Max 3 active loans per user. 4th attempt returns HTTP 400 'Borrow limit of 3 reached'.

Core Requirements - Non-Functional (Phase 1)

NFR-1

Security

Passwords are hashed with bcrypt or argon2id before storage. Plain-text passwords are never written to the database or logs.

NFR-2

Security

Admin-only API routes are protected by a server-side role check on every request. Client-side hiding of UI elements alone is not sufficient protection.

NFR-3

Security

JWTs expire after 30 minutes of inactivity. Refresh tokens rotate on each use. HTTPS is enforced in production; HTTP-only cookies are used where applicable.

NFR-4

Security

All user inputs are sanitized via parameterized SQL statements and output-encoded to prevent XSS. The OWASP Top 10 is used as an implementation checklist.

NFR-9

Usability

A first-time user can complete registration and view the catalog within 3 minutes with no prior training. Verified by a structured usability walkthrough before release.

NFR-10

Accessibility

UI follows WCAG 2.1 AA: all form fields have labels, and interactive elements are keyboard-navigable.

Core Requirements - Non-Functional (Phase 2)

NFR-5

Performance

95% of catalog and borrow/return API requests respond within 2 seconds under normal class-sized load (up to 25 concurrent users). Verified with manual load testing using concurrent curl requests.

NFR-6

Scalability

The system supports at least 25 concurrent users without failure or data corruption. The PostgreSQL connection pool is configured with a pool size appropriate for this load.

NFR-7

Atomicity

All borrow and return database operations are wrapped in BEGIN/COMMIT transactions. If any query within the transaction fails, a ROLLBACK is executed automatically, leaving the database in a consistent state with no partial updates to available_copies or transaction status.

Implementation - High Level Overview

Frontend
HTML / CSS / JavaScript

Backend
Node.js + Express

Auth Layer
JWT tokens + bcrypt

Database
PostgreSQL + Connection Pool

Key Implementation Details

- borrowController.js handles all loan logic with atomic DB transactions
- All borrow/return logic runs on a single pooled connection so the BEGIN, the INSERT/UPDATE on books, and the COMMIT all happen on the same connection, without that, the queries could land on different connections and the -transaction wouldn't be all or nothing
- 3-book limit enforced server-side before transaction begins
- JWT middleware protects all borrow/return routes
- Duplicate loan detection via active transaction lookup
- GET /api/borrow/my returns full loan history with due_date & status
- Frontend disables Borrow button when available_copies = 0

Code Inspection Peer Review Feedback

What they caught

- the "3" in borrow limit from the borrowcontroller wasn't put anywhere — fixed by pulling it out as MAX_BORROW_LIMIT at the top of the file
- Missing path comments at the top of route files
- Falsy checks that didn't account for "0" being a valid value
- Hardcoded color values in the frontend instead of referencing CSS variables

What we did

- Fixed all four. None of them were correctness bugs but they were the kind of thing that makes code harder to maintain six months later.

Takeaway

- new eyes would catch stuff you stop seeing in your own code. We'd do peer review earlier next time, not just once.

Testing Experiences

How we tested

- 10 endpoints exercised with curl, hitting the route, controller, and DB
- Each borrow/return verified twice, the first time using the API response, and another by querying the database directly to confirm the available_copies actually changed

Highlights

- Caught a real bug: pool.query('BEGIN') doesn't pin to a single connection, so our transaction wasn't packaged together Fixed this by checking out a client with pool.connect() and running BEGIN/COMMIT/ROLLBACK on that same client.
- Verified by promoting a user to admin in the DB, re-logging in, and watching the admin routes go from 403 to 200.

What we'd add with more time

- Automated test suite, Jest and supertest so we don't have to re-run curl by hand every time
- Load testing to actually verify the 25-user requirement instead of just claiming it

Limitations and Constraints

01

Scope Limits

- No email notifications for due dates, returns, or account activity.
- No fines or fees system — overdue loans are tracked but not penalized.
- No reservations, holds, or waitlists for checked-out books.
- Admin tooling focused on core CRUD; no reporting dashboard.

02

Deployment & Infrastructure

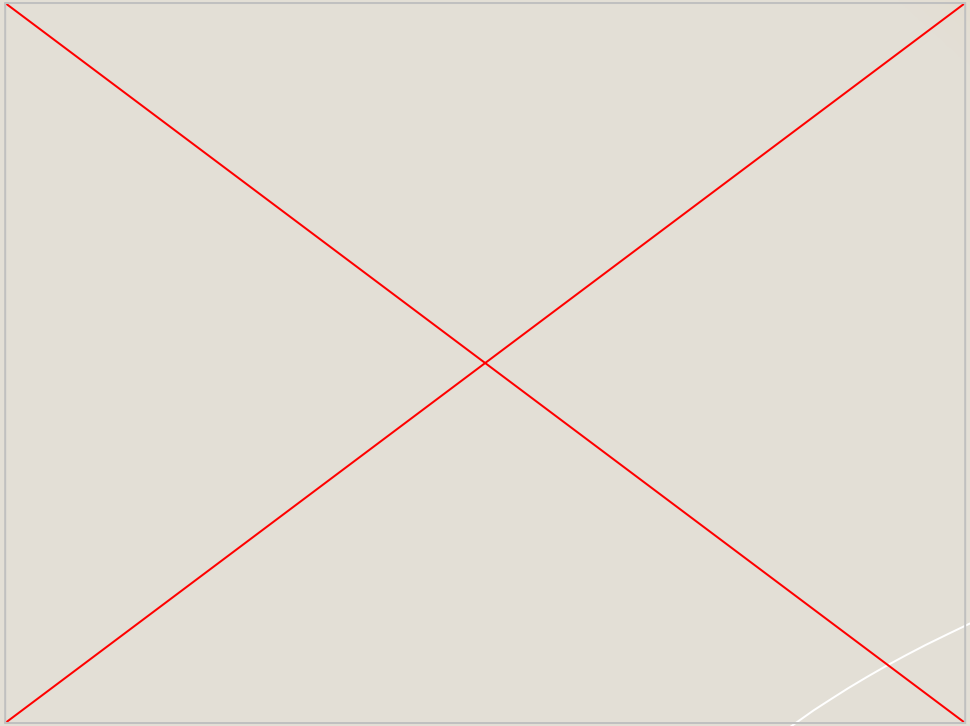
- Single-server architecture — no load balancer or failover.
- Not deployed to a hosted environment; runs locally with one PostgreSQL instance.
- No CI/CD pipeline — deploys are manual.
- No monitoring, alerting, or centralized logging in place.

03

Time & Team Size

- Five-person team working within a single semester.
- Feature set scoped to Phase 1–3 requirements to stay on schedule.
- Manual testing only — no time to build an automated suite.
- Usability and load claims verified by structured walkthroughs, not formal studies.

Demo



https://drive.google.com/file/d/11a9iRViXPmoWZMrcGN8pOANtYdTRHuRX/view?usp=sharing

Conclusion

What we delivered

A working web-based Library Management System built on Node.js, Express, and PostgreSQL, covering authentication, a browsable catalog, and a transactional borrow/return flow with a hard 3-book limit.

All Phase 1 and Phase 2 requirements — functional and non-functional — are implemented, with borrow/return wrapped in atomic DB transactions and JWT-protected routes enforcing role-based access.

Met every core requirement

FR-1 through FR-16 and NFR-1 through NFR-7, NFR-9, NFR-10 — all verified.

Caught and fixed a real bug

Transactions were not pinned to a single pooled connection — found during manual DB testing.

Biggest takeaway

Peer review catches things you stop seeing in your own code. Next time: review earlier, not just once.

Thank you — questions?

References

[1] Node.js Documentation

OpenJS Foundation. Node.js Docs. https://nodejs.org/en/docs

[2] Express.js Documentation

OpenJS Foundation. Express — Fast, unopinionated, minimalist web framework for Node.js. https://expressjs.com

[3] PostgreSQL Documentation

The PostgreSQL Global Development Group. PostgreSQL 16 Documentation. https://www.postgresql.org/docs/

[4] node-postgres (pg)

Carlson, B. node-postgres: PostgreSQL client for Node.js. https://node-postgres.com

[5] jsonwebtoken

Auth0. jsonwebtoken — JWT implementation for Node.js. https://github.com/auth0/node-jsonwebtoken